

Dossier Professionnel

**Titre Professionnel : Concepteur Développeur d'Applications
(CDA)**

Candidat : LE PLAT Jonathan

Date de naissance : 27/07/1991

Organisme de formation : La Plateforme_

Année : 2026

Activité-type N°1 : Développer une application sécurisée

Exemple n°1 : Développement du système d'authentification sécurisé de Promptimo

1. Tâches et opérations effectuées

Dans le cadre du développement de Promptimo, un SaaS B2B destiné aux agents immobiliers, j'ai conçu et développé le système d'authentification de l'application en m'appuyant sur NextAuth v5 (beta) en stratégie JWT.

Promptimo repose sur un modèle d'accès par invitation : seuls les agents pré-enregistrés après paiement peuvent se connecter, sans possibilité d'auto-inscription. Pour cela j'ai :

- Implémenté un adaptateur Prisma personnalisé (custom adapter) sur mon propre modèle Agent, remplaçant l'adaptateur officiel pour mapper directement le modèle existant sans dupliquer les données utilisateurs
- Configuré un flux de connexion sans mot de passe (magic link) via l'API Resend, générant un lien de connexion unique à durée limitée (24h), validé par un token haché SHA-256
- Sécurisé le token JWT avec chiffrement JWE (JSON Web Encryption) via la bibliothèque jose, embarquant les champs personnalisés agentId, agencyId et role
- Protégé toutes les routes de l'espace connecté via le middleware Next.js, vérifiant la présence et la validité du token JWT à chaque requête
- Développé un système de génération de magic link côté backend pour le flux post-paiement Stripe, permettant l'activation automatique du compte après validation du paiement

2. Moyens utilisés

Next.js 16 (App Router), NextAuth v5 beta, TypeScript, Prisma v7, PostgreSQL, Resend (envoi d'emails transactionnels), JWT/JWE (jose), middleware Next.js, Git.

3. Collaboration

Développé en autonomie dans le cadre de mon projet Promptimo.

4. Contexte

Organisme : Auto-entreprise

Période : Mars 2026 – Juillet 2026

Exemple n°2 : Développement du flux d'inscription et de paiement sécurisé via Stripe

1. Tâches et opérations effectuées

Pour le processus d'inscription de Promptimo, j'ai développé un flux complet d'inscription et de paiement, articulé autour de Stripe Checkout et d'un système de webhooks sécurisés. Le flux se déroule en trois étapes distinctes :

- Soumission du formulaire d'inscription : validation SIRET via l'API officielle INSEE (algorithme de Luhn + vérification dans le registre), vérification d'unicité de l'email, stockage temporaire dans une table PendingRegistration avec date d'expiration (24h)
- Redirection vers Stripe Checkout pour le paiement de l'abonnement mensuel (49 €/mois, mode subscription)

- Réception du webhook stripe checkout.session.completed côté serveur, déclenchant la création de l'agence et de l'agent, puis l'envoi d'un magic link d'activation par email

Pour sécuriser ce flux j'ai :

- Vérifié la signature HMAC SHA-256 de chaque webhook entrant via la clé secrète Stripe Webhook, rejetant toute requête non authentifiée
- Utilisé des transactions Prisma pour garantir l'atomicité de la création Agency + Agent (tout ou rien en cas d'erreur)
- Nettoyé automatiquement les PendingRegistrations expirées pour éviter les doublons et respecter la contrainte d'unicité sur l'email
- Géré tous les cas d'erreur : email déjà utilisé (409), SIRET invalide, paiement annulé (redirection vers la page d'inscription)

2. Moyens utilisés

Next.js 16 (API Routes), Stripe SDK v22, Zod (validation du schéma côté serveur), Prisma v7, PostgreSQL, Resend, TypeScript, Git.

3. Collaboration

Développé en autonomie dans le cadre de mon projet Promptimo.

4. Contexte

Organisme : Auto-entreprise

Période : Mars 2026 – Juillet 2026

Exemple n°3 : Développement de la génération d'annonces immobilières par IA avec streaming SSE

1. Tâches et opérations effectuées

L'une des fonctionnalités centrales de Promptimo est la génération automatique d'annonces immobilières via l'API Anthropic (modèle claude-sonnet-4-20250514). J'ai développé cette fonctionnalité en streaming Server-Sent Events (SSE) afin d'offrir une expérience utilisateur fluide, sans bloquer l'interface pendant la génération.

Concrètement j'ai :

- Développé une API route Next.js (/api/listings/stream) recevant l'identifiant du bien, récupérant ses caractéristiques depuis la base de données (type, surface, prix, ville, équipements, notes agent), et construisant un prompt structuré à destination du modèle
- Implémenté le streaming de la réponse Anthropic en SSE : le texte est transmis au navigateur au fur et à mesure de sa génération via ReadableStream, sans attendre la fin de la réponse complète
- Développé l'interface React consommant ce flux avec l'API Streams (getReader/read loop), affichant le texte en temps réel avec un curseur clignotant animé
- Sécurisé la route par vérification de session JWT : seul l'agent authentifié et propriétaire du bien peut déclencher la génération
- Sauvegardé l'annonce générée en base de données pour la réutiliser sans refaire appel à l'API, et développé une fonctionnalité de régénération à la demande

2. Moyens utilisés

Next.js 16 (App Router, API Routes), Anthropic SDK (claude-sonnet-4-20250514), Server-Sent Events (SSE), ReadableStream, React 19, TypeScript, Prisma v7, PostgreSQL, Git.

3. Collaboration

Développé en autonomie dans le cadre de mon projet Promptimo.

4. Contexte

Organisme : Auto-entreprise

Période : Mars 2026 – Juillet 2026

Activité-type N°2 : Concevoir et développer une application sécurisée organisée en couches

Exemple n°1 : Conception de l'architecture en couches de Promptimo

1. Tâches et opérations effectuées

Pour Promptimo, j'ai conçu une architecture full-stack organisée en couches claires, en m'appuyant sur les conventions du framework Next.js 16 avec l'App Router. L'application est structurée selon trois niveaux de responsabilité bien distincts :

- Couche présentation (UI) : composants React Server Components (RSC) et Client Components, organisés par fonctionnalité sous `src/app/(protected)/`. Les RSC effectuent les lectures de données directement via Prisma sans passer par une API intermédiaire, réduisant la latence et le nombre d'appels réseau
- Couche API (API Routes) : routes Next.js sous `src/app/api/`, responsables de toutes les mutations (création, mise à jour, suppression), de l'authentification des requêtes (vérification session JWT), de la validation des données entrantes (Zod) et des interactions avec les services externes (Stripe, Anthropic, Resend)
- Couche données : accès base de données centralisé via Prisma v7 avec adaptateur pg, utilisant un singleton global pour éviter la multiplication des connexions en développement

Cette organisation garantit la séparation des responsabilités, facilite les tests unitaires et d'intégration des API routes de manière isolée, et maintient la scalabilité du code.

2. Moyens utilisés

Next.js 16 (App Router, RSC, API Routes), TypeScript, Prisma v7 (adaptateur pg), Zod, React 19, Git.

3. Collaboration

Développé en autonomie dans le cadre de mon projet Promptimo.

4. Contexte

Organisme : Auto-entreprise

Période : Mars 2026 – Juillet 2026

Exemple n°2 : Modélisation et conception de la base de données relationnelle de Promptimo

1. Tâches et opérations effectuées

Pour Promptimo, j'ai conçu un schéma de base de données relationnel répondant aux besoins métier d'une agence immobilière multi-agents. La modélisation s'articule autour de cinq entités principales :

- Agency : l'agence immobilière, tenant principal du système (approche multi-tenant par ligne)
- Agent : l'utilisateur de l'application, rattaché à une agence avec un rôle (admin ou agent)
- Client : le contact client, rattaché à l'agent qui le gère
- Property : le bien immobilier, rattaché à un agent et optionnellement à un client propriétaire/vendeur
- GeneratedEmail : les emails générés par l'IA, liés à un agent, un client et un bien

J'ai veillé à :

- Garantir l'isolation des données entre agences : chaque entité remonte à une Agency via des clés étrangères, empêchant qu'un agent d'une agence accède aux données d'une autre
- Gérer les tokens de vérification NextAuth dans un modèle VerificationToken dédié, avec contrainte d'unicité sur (identifier, token)
- Utiliser les migrations Prisma pour versionner rigoureusement l'évolution du schéma et déployer en production via prisma migrate deploy
- Gérer les inscriptions en attente dans une table PendingRegistration avec date d'expiration, permettant un nettoyage automatique des inscriptions abandonnées

2. Moyens utilisés

Prisma v7 (schema.prisma, migrations), PostgreSQL 18, TypeScript, Git.

3. Collaboration

Développé en autonomie dans le cadre de mon projet Promptimo.

4. Contexte

Organisme : Auto-entreprise

Période : Mars 2026 – Juillet 2026

Exemple n°3 : Développement de l'interface utilisateur de Promptimo

1. Tâches et opérations effectuées

J'ai développé l'intégralité de l'interface utilisateur de Promptimo, de la page d'accueil publique jusqu'à l'espace connecté, en visant une expérience soignée à destination des professionnels de l'immobilier.

Pour la partie publique (landing page, pricing, inscription) :

- Conçu une page d'accueil mettant en valeur les fonctionnalités IA avec une typographie affirmée et des animations CSS
- Développé une page pricing épurée, offre unique à 49 €/mois, avec un parcours de conversion direct vers le formulaire d'inscription
- Intégré un formulaire d'inscription multi-champs avec validation en temps réel : algorithme de Luhn pour le SIRET, confirmation email, formatage automatique du téléphone, validation côté client avec react-hook-form + Zod

Pour l'espace connecté :

- Développé une navigation responsive : sidebar fixe sur desktop, bouton flottant avec menu plein écran sur mobile (position ergonomique bas-droite pour pouce droit), thème clair/sombre
- Implémenté la gestion des clients : liste paginée, dialog de création, filtres, compteurs de biens et d'emails associés
- Développé le formulaire de saisie d'un bien immobilier avec sélection visuelle du type de bien et du mode de transaction (vente/location)
- Intégré une vue carte Leaflet affichant les biens géolocalisés via l'API Nominatim (géocodage automatique à la saisie), avec popups de détails et bottom sheet responsive sur mobile

2. Moyens utilisés

Next.js 16 (App Router), React 19, TypeScript, Tailwind CSS v4, shadcn/ui, Leaflet, react-leaflet, react-hook-form, Zod, API Nominatim (géocodage), Git.

3. Collaboration

Développé en autonomie dans le cadre de mon projet Promptimo.

4. Contexte

Organisme : Auto-entreprise

Période : Mars 2026 – Juillet 2026

Activité-type N°3 : Préparer le déploiement d'une application sécurisée

Exemple n°1 : Mise en place de la stratégie de tests de Promptimo (Vitest + Playwright)

1. Tâches et opérations effectuées

Pour garantir la fiabilité des fonctionnalités critiques de Promptimo, j'ai mis en place une stratégie de tests automatisés à deux niveaux : tests unitaires et d'intégration avec Vitest, et tests End-to-End (E2E) avec Playwright.

Tests unitaires et d'intégration (Vitest) :

- Tests unitaires de la logique de génération de prompt IA (fonction buildPrompt), validant différents profils de biens pour garantir la qualité du prompt transmis au modèle Anthropic
- Tests d'intégration de POST /api/register : validation Zod, détection des doublons email, vérification de la création correcte de la PendingRegistration, appel à l'API Stripe Checkout
- Tests d'intégration de POST /api/webhooks/stripe : vérification de la signature HMAC, création atomique Agency + Agent, génération du token de vérification, envoi de l'email d'activation — services externes (Stripe, Resend) mockés via vi.mock()

Tests End-to-End (Playwright) :

- Mise en place d'un global setup créant un agent de test isolé en base de données (agence dédiée, email e2e@playwright.test), encodant un JWT d'authentification valide et le sauvegardant comme état de session Playwright
- Global teardown supprimant systématiquement l'agence et l'agent de test après chaque exécution, garantissant l'idempotence des tests
- 4 scénarios E2E couvrant le parcours complet : affichage de la liste clients, création d'un client via dialog, création d'un bien immobilier et redirection vers la page annonce, génération d'une annonce IA (avec interception SSE mockée)
- Connexion à la base de données de production via tunnel SSH (localhost:5433 → VPS:5432), permettant de tester contre de vraies données sans infrastructure supplémentaire

2. Moyens utilisés

Vitest, @testing-library/react, vi.mock() (Stripe, Resend), Playwright, PostgreSQL (VPS via tunnel SSH), TypeScript, Git.

3. Collaboration

Tests développés en autonomie dans le cadre de mon projet Promptimo.

4. Contexte

Organisme : Auto-entreprise

Période : Mars 2026 – Juillet 2026

Exemple n°2 : Déploiement de Promptimo sur VPS avec PostgreSQL, PM2 et Nginx

1. Tâches et opérations effectuées

J'ai déployé Promptimo sur un VPS Ionos S+ (Ubuntu 26.04) en configurant l'ensemble de l'environnement de production depuis zéro, sans utiliser de conteneurisation, pour garder un contrôle direct sur chaque composant.

Les étapes du déploiement initial :

- Installation et configuration de PostgreSQL 18 : création de la base de données promptimo, création d'un utilisateur applicatif dédié avec les droits nécessaires (CREATE, CONNECT, USAGE), vérification du service systemd
- Installation de Node.js 24 via nvm et de pnpm, clonage du dépôt Git depuis GitHub et positionnement sur la branche main
- Création du fichier .env de production directement sur le serveur (hors Git), renseignant DATABASE_URL, AUTH_SECRET, clés Stripe, Resend et Anthropic
- Déploiement des migrations Prisma en production (pnpm prisma migrate deploy), génération du client Prisma (pnpm prisma generate), puis build Next.js (pnpm build)
- Lancement de l'application via PM2 en mode daemon, configuré pour redémarrer automatiquement en cas de crash ou de reboot du serveur
- Configuration de Nginx en reverse proxy : redirection des requêtes HTTP entrantes (port 80) vers le port 3000 de l'application Node.js

Les déploiements suivants sont automatisés via le pipeline CI/CD GitHub Actions (voir Exemple n°4) : chaque push sur la branche main déclenche les tests E2E puis le déploiement automatique sur le VPS.

L'application est accessible à l'adresse <http://82.165.114.226>, avec mise en place prévue d'un nom de domaine et d'un certificat SSL/TLS via Let's Encrypt (Certbot).

2. Moyens utilisés

VPS Ionos S+ (Ubuntu 26.04), PostgreSQL 18, Node.js 24 (nvm), pnpm, Prisma v7 (migrations deploy), PM2 (process manager), Nginx (reverse proxy), Git, SSH, GitHub Actions.

3. Collaboration

Déploiement réalisé en autonomie dans le cadre de mon projet Promptimo.

4. Contexte

Organisme : Auto-entreprise

Période : Mars 2026 – Juillet 2026

Exemple n°3 : Gestion des environnements et sécurisation des secrets de Promptimo

1. Tâches et opérations effectuées

Pour Promptimo, j'ai mis en place une séparation stricte entre les environnements de développement, de test et de production, garantissant l'isolation des données et la non-exposition des secrets.

J'ai :

- Créé un fichier .env pour le développement local, pointant sur le VPS via tunnel SSH (ssh -L 5433:localhost:5432), avec les clés Stripe en mode test

- Créé un fichier `.env.test` pour les tests automatisés Playwright et Vitest, connecté directement au VPS PostgreSQL via tunnel SSH (`DATABASE_URL` pointant sur `localhost:5433`), avec des placeholders pour les services externes non nécessaires aux tests (Stripe, Resend, Anthropic)
- Configuré le `.gitignore` pour exclure tous les fichiers `.env*` du dépôt Git dès la création du projet, évitant toute exposition accidentelle de secrets (clés API, `AUTH_SECRET`, mots de passe de base de données)
- Créé le fichier `.env` de production directement sur le VPS via SSH, jamais versionné ni transmis par un canal non sécurisé
- Stocké les secrets nécessaires au CI/CD (`VPS_SSH_KEY`, `DATABASE_URL_TEST`, `AUTH_SECRET`) dans les GitHub Actions Secrets, accessibles dans le workflow sans jamais apparaître en clair dans le code source
- Retiré du suivi Git des fichiers sensibles ajoutés trop tardivement au `.gitignore`, via `git rm --cached`, pour assainir l'historique sans supprimer les fichiers locaux

2. Moyens utilisés

Git (`.gitignore`, `git rm --cached`, branches), fichiers `.env` (séparation dev/test/prod), tunnel SSH, VPS Ionos, GitHub Actions Secrets, Stripe (modes test/live).

3. Collaboration

Configuration réalisée en autonomie dans le cadre de mon projet Promptimo.

4. Contexte

Organisme : Auto-entreprise

Période : Mars 2026 – Juillet 2026

Exemple n°4 : Mise en place du pipeline CI/CD avec GitHub Actions

1. Tâches et opérations effectuées

Pour automatiser les tests et le déploiement de Promptimo, j'ai mis en place un pipeline CI/CD complet avec GitHub Actions, déclenché automatiquement à chaque push sur la branche main.

Le pipeline se déroule en deux phases distinctes :

Phase 1 — Tests automatisés :

- Installation des dépendances (`pnpm install`) et génération du client Prisma (`pnpm prisma generate`), étape indispensable car le client généré n'est pas versionné dans Git
- Configuration de la clé SSH depuis les GitHub Secrets et ajout du VPS dans les `known_hosts` via `ssh-keyscan`
- Ouverture d'un tunnel SSH (`ssh -f -N -L 5433:localhost:5432`) permettant aux tests de se connecter à la base de données PostgreSQL du VPS depuis le runner GitHub Actions
- Reconstruction du fichier `.env.test` à partir des GitHub Secrets (`DATABASE_URL_TEST`, `AUTH_SECRET`), sans jamais stocker ces valeurs dans le code source
- Installation de Chromium via `playwright install` et exécution des 4 tests E2E

Phase 2 — Déploiement automatique (conditionnel au succès des tests) :

- Connexion SSH au VPS et bascule sur la branche main (`git checkout main && git pull origin main`)
- Build de l'application Next.js (`pnpm build`) puis redémarrage via PM2 (`pm2 restart promptimo`)
- En cas d'échec des tests, le déploiement est annulé (condition `if: success()`), protégeant la production

2. Moyens utilisés

GitHub Actions (ubuntu-latest runner), pnpm/action-setup, actions/setup-node, SSH (tunnel + déploiement), Playwright, PM2, Git, GitHub Secrets.

3. Collaboration

Pipeline CI/CD mis en place en autonomie dans le cadre de mon projet Promptimo.

4. Contexte

Organisme : Auto-entreprise

Période : Mars 2026 – Juillet 2026

Annexes — Extraits de code commentés

Les extraits ci-dessous illustrent les points techniques les plus significatifs du projet Promptimo. Ils sont présentés avec des annotations expliquant les choix d'implémentation.

Annexe 1 : Authentification NextAuth v5 — Adaptateur personnalisé et JWT

Promptimo n'utilise pas le modèle User standard de NextAuth mais son propre modèle Agent. J'ai donc implémenté un adaptateur personnalisé qui mappe les méthodes NextAuth sur la table Agent, et un callback JWT qui enrichit le token avec les champs métier (agentId, agencyId, role).

```
// src/lib/auth.ts - Adaptateur personnalisé
// createUser est volontairement absent : seuls les agents invités peuvent se
// connecter.
// Toute tentative d'auto-inscription lève une erreur 403 explicite.
const agentAdapter: Adapter = {
  async createUser(_data: AdapterUser) {
    throw new Error('Auto sign-up is disabled. Contact your agency admin.')
  },

  // getUserByEmail cherche dans la table Agent, pas User.
  // On inclut l'agency pour récupérer agencyId en une seule requête.
  async getUserByEmail(email: string) {
    const agent = await prisma.agent.findUnique({
      where: { email },
      include: { agency: true },
    })
    if (!agent) return null
    return toAdapterUser(agent)
  },
}

// src/lib/auth.ts - Callbacks JWT et Session
// Callback jwt : au premier login, "user" contient les données de l'adaptateur.
// On les copie dans le token pour les rendre disponibles côté serveur.
callbacks: {
  async jwt({ token, user }) {
    if (user) {
      const u = user as AdapterUser & { agencyId?: string; role?: string }
      token.agentId = u.id
      token.agencyId = u.agencyId
      token.role = u.role
      token.name = u.name
    }
    return token
  },

  // Callback session : expose les champs du token aux composants React
  // via useSession() ou auth() côté serveur.
  async session({ session, token }) {
    session.user.id = token.agentId as string
    session.user.agencyId = token.agencyId as string
    session.user.role = token.role as string
    return session
  },
}
```

Annexe 2 : Webhook Stripe — Vérification HMAC et création atomique

Le webhook Stripe est le point d'entrée critique du flux d'inscription. Il doit vérifier l'authenticité de la requête, créer l'agence et l'agent de façon atomique, puis envoyer le magic link d'activation. Chaque étape est sécurisée.

```
// src/app/api/webhooks/stripe/route.ts - Vérification de la signature HMAC
// stripe.webhooks.constructEvent vérifie la signature HMAC-SHA256 de la requête.
// Si la signature ne correspond pas à STRIPE_WEBHOOK_SECRET, une erreur est
levée
// et on répond 400 - protégeant contre les appels frauduleux.
let event: Stripe.Event
try {
  event = stripe.webhooks.constructEvent(body, sig,
process.env.STRIPE_WEBHOOK_SECRET!)
} catch {
  return NextResponse.json({ error: 'Invalid signature' }, { status: 400 })
}
```

```
// src/app/api/webhooks/stripe/route.ts - Création atomique Agency + Agent
// prisma.agency.create avec agents.create imbriqué garantit l'atomicité :
// si la création de l'Agent échoue, l'Agency n'est pas créée non plus.
// Pas besoin de transaction explicite : Prisma le gère en interne.
const agency = await prisma.agency.create({
  data: {
    name: pending.agencyName,
    slug,
    agents: {
      create: {
        email: pending.email,
        name: `${pending.firstName} ${pending.lastName}`,
        // role non spécifié → valeur par défaut "agent" (schema.prisma
@default)
      },
    },
  },
})
```

```
// src/app/api/webhooks/stripe/route.ts - Génération du magic link sécurisé
// Le token est généré aléatoirement (32 octets), puis haché SHA-256
// avant stockage en base (même algorithme que NextAuth v5).
// L'URL magic link contient le token en clair - seul le hash est en BDD.
function hashToken(token: string): string {
  return createHash('sha256')
    .update(`${token}${process.env.AUTH_SECRET}`)
    .digest('hex')
}
```

```
const plainToken = randomBytes(32).toString('hex')
const hashedToken = hashToken(plainToken)
const expires = new Date(Date.now() + 24 * 60 * 60 * 1000) // 24h

await prisma.verIFICATION_TOKEN.create({
  data: { identifier: pending.email, token: hashedToken, expires },
})
```

Annexe 3 : Génération d'annonce en streaming SSE (Anthropic API)

La route de streaming est la pièce centrale de la proposition de valeur de Promptimo. Elle reçoit l'identifiant d'un bien, construit un prompt, appelle l'API Anthropic en mode streaming, retransmet chaque fragment de texte au navigateur en temps réel via ReadableStream, puis sauvegarde le résultat complet en base.

```
// src/app/api/listings/stream/route.ts - Sécurité et streaming
// Double vérification de sécurité :
// 1. Session JWT valide (agent connecté)
// 2. Le bien appartient bien à cet agent (agentId dans le WHERE)
const session = await auth()
if (!session?.user?.id)
  return NextResponse.json({ error: 'Non autorisé' }, { status: 401 })

const property = await prisma.property.findUniqueOrThrow({
  where: { id: propertyId, agentId: session.user.id },
})

// src/app/api/listings/stream/route.ts - ReadableStream + Anthropic SDK
// ReadableStream transmet les fragments de texte au fur et à mesure
// sans attendre la fin de la réponse Anthropic.
// Le client React lit ce flux avec getReader() et affiche chaque chunk.
const stream = new ReadableStream({
  async start(controller) {
    const anthropicStream = anthropic.messages.stream({
      model: 'claude-sonnet-4-20250514',
      max_tokens: 1024,
      messages: [{ role: 'user', content: prompt }],
    })

    for await (const event of anthropicStream) {
      // On ne traite que les deltas de texte, on ignore les autres events
      if (event.type === 'content_block_delta'
        && event.delta.type === 'text_delta') {
        fullText += event.delta.text
        controller.enqueue(encoder.encode(event.delta.text))
      }
    }

    // Sauvegarde en BDD une fois le stream terminé
    // → réutilisable sans refaire appel à l'API
    await prisma.property.update({
      where: { id: propertyId },
      data: { generatedListing: fullText },
    })
    controller.close()
  },
})

return new Response(stream, {
  headers: { 'Content-Type': 'text/plain; charset=utf-8' },
})
```

Annexe 4 : Pipeline CI/CD GitHub Actions

Le fichier ci-dessous définit le workflow GitHub Actions déclenché à chaque push sur main. Il exécute les tests E2E en utilisant un tunnel SSH vers la base de données du VPS, et déploie automatiquement en cas de succès.

```
// .github/workflows/ci-cd.yml
name: CI/CD

on:
  push:
    branches: [main]

jobs:
  test-and-deploy:
    runs-on: ubuntu-latest
    timeout-minutes: 15

    steps:
      - uses: actions/checkout@v4

      # Installation de pnpm 10 et Node.js 20 avec cache
      - uses: pnpm/action-setup@v4
        with:
          version: 10
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'pnpm'

      - name: Install dependencies
        run: pnpm install

      # Indispensable : le client Prisma généré n'est pas dans Git (.gitignore)
      - name: Generate Prisma client
        run: pnpm prisma generate

      # Configuration SSH depuis les GitHub Secrets
      - name: Setup SSH
        run: |
          mkdir -p ~/.ssh
          echo "${{ secrets.VPS_SSH_KEY }}" > ~/.ssh/id_rsa
          chmod 600 ~/.ssh/id_rsa
          ssh-keyscan -H 82.165.114.226 >> ~/.ssh/known_hosts

      # Tunnel SSH : localhost:5433 → VPS PostgreSQL:5432
      # Permet aux tests de se connecter à la BDD de production depuis CI
      - name: Open SSH tunnel to VPS database
        run: ssh -f -N -L 5433:localhost:5432 root@82.165.114.226

      # Reconstruction du .env.test depuis les GitHub Secrets
      # (jamais stocké dans Git)
      - name: Write .env.test
        run: |
          echo "DATABASE_URL=${{ secrets.DATABASE_URL_TEST }}" >> .env.test
          echo "AUTH_SECRET=${{ secrets.AUTH_SECRET }}" >> .env.test
          echo "NEXTAUTH_URL=http://localhost:3000" >> .env.test

      - name: Install Playwright browsers
        run: pnpm exec playwright install --with-deps chromium

      - name: Run E2E tests
        run: pnpm test:e2e
```

```
# Déploiement conditionnel : uniquement si tous les tests passent
- name: Deploy to VPS
  if: success()
  run: |
    ssh root@82.165.114.226 \
      "cd /var/www/promptimo \
        && git checkout main && git pull origin main \
        && pnpm build && pm2 restart promptimo"
```